

Python Full Stack Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

1. Programming & Software Development Fundamentals

- Introduction to software development
- How web applications work
- Client-server architecture
- Frontend vs backend development
- Three-tier architecture
- HTTP and HTTPS fundamentals
- Request-response lifecycle
- Static vs dynamic websites
- REST APIs
- Development, staging, and production environments
- IDE setup: VS Code / PyCharm
- Terminal and command-line fundamentals
- Basic Git and GitHub introduction

Part I — Python Programming

2. Python Fundamentals

- Introduction to Python
- Python installation and environment setup
- Python syntax
- Variables and identifiers
- Keywords
- Comments
- Input and output
- Data types
- Type conversion
- Operators
- Arithmetic
- Assignment
- Comparison
- Logical
- Bitwise
- Membership
- Identity
- Operator precedence

Control Flow

- if
- if-else
- if-elif-else
- Nested conditions
- match-case
- for loops
- while loops
- Nested loops

- break
- continue
- pass

3. Python Data Structures

Strings

- Creating strings
- String indexing
- Slicing
- String methods
- String formatting
- f-strings
- String immutability

Lists

- Creating lists
- Indexing and slicing
- List methods
- Adding/removing elements
- Sorting
- Nested lists
- List comprehension

Tuples

- Tuple creation
- Packing and unpacking
- Tuple methods
- List vs tuple

Sets

- Creating sets
- Set operations
- Union
- Intersection
- Difference
- Symmetric difference

Dictionaries

- Key-value pairs
- Adding/updating data
- Dictionary methods
- Iterating dictionaries
- Nested dictionaries
- Dictionary comprehension

4. Functions in Python

- Defining functions
- Calling functions
- Function parameters
- Positional arguments
- Keyword arguments
- Default arguments

- Variable-length arguments
- `*args`
- `**kwargs`
- Return statement
- Multiple return values
- Scope of variables
- Local/global variables
- `global`
- `nonlocal`
- Lambda expressions
- Recursive functions
- Higher-order functions
- `map()`
- `filter()`
- `reduce()`
- Closures
- Decorators
- Generators
- `yield`

5. Object-Oriented Programming with Python

OOP Fundamentals

- Class
- Object
- Attributes
- Methods
- Constructors
- `__init__()`
- `self`

OOP Principles

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

Advanced OOP

- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- Hierarchical inheritance
- Method overriding
- Method overloading concepts
- Abstract classes
- Interfaces using ABC
- Instance methods
- Class methods
- Static methods
- Properties
- Getter/setter methods
- Magic/dunder methods
- Composition

- Aggregation
- Association
- MRO — Method Resolution Order
- SOLID principles introduction

6. Exception Handling

- Types of errors
- Syntax errors
- Runtime errors
- Logical errors
- Exception handling
- try
- except
- else
- finally
- Multiple exceptions
- Raising exceptions
- Custom exceptions
- Exception hierarchy
- Best practices for error handling

7. File Handling

- Working with files
- Text files
- Binary files
- Reading files
- Writing files
- Appending files
- File modes
- with statement
- Working with CSV
- Working with JSON
- Serialization
- Deserialization
- Working with directories
- os
- pathlib
- File uploads/download concepts

8. Python Modules, Packages & Environment Management

- Modules
- Packages
- Import system
- Creating custom modules
- Standard Python library
- pip
- PyPI
- Installing third-party packages
- requirements.txt
- Virtual environments
- venv

- Environment variables
- .env files
- Dependency management
- Project folder structure

9. Advanced Python

- Iterators
- Generators
- Decorators
- Closures
- Context managers
- Type hints
- Dataclasses
- Enumerations
- Regular expressions
- Date and time
- Collections module
- Functional programming concepts
- Shallow copy vs deep copy
- Mutable vs immutable objects
- Memory management basics
- Garbage collection
- Logging
- Debugging
- Python coding standards
- PEP 8

10. Data Structures & Algorithms with Python

Data Structures

- Arrays
- Lists
- Stack
- Queue
- Deque
- Hash table
- Linked list
- Trees
- Binary trees
- Binary search trees
- Heap
- Graphs

Algorithms

- Searching
- Linear search
- Binary search
- Sorting
- Bubble sort
- Selection sort
- Insertion sort
- Merge sort
- Quick sort

- Recursion
- Two-pointer technique
- Sliding window
- Hashing
- Basic dynamic programming

Complexity Analysis

- Time complexity
- Space complexity
- Big-O notation

Part II — Database Development

11. SQL & Relational Database Fundamentals

Recommended: PostgreSQL or MySQL

- Introduction to databases
- DBMS vs RDBMS
- Tables, rows and columns
- Primary keys
- Foreign keys
- Candidate keys
- Constraints
- Data types

SQL

- CREATE
- ALTER
- DROP
- INSERT
- UPDATE
- DELETE
- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING
- Aggregate functions
- String functions
- Date functions
- Joins
- Inner join
- Left join
- Right join
- Full join
- Self join
- Subqueries
- Views
- Indexes
- Stored procedures concepts
- Transactions
- ACID properties
- Normalization

- Database design
- ER diagrams
- One-to-one relationships
- One-to-many relationships
- Many-to-many relationships

12. NoSQL & MongoDB

- SQL vs NoSQL
- MongoDB concepts
- Databases
- Collections
- Documents
- BSON
- CRUD operations
- Query operators
- Filtering
- Sorting
- Aggregation
- Indexes
- Embedded documents
- References
- MongoDB Atlas
- Connecting Python applications with MongoDB

Part III — Web Fundamentals

13. HTML5

- Introduction to HTML
- HTML document structure
- Elements and attributes
- Headings
- Paragraphs
- Text formatting
- Links
- Images
- Lists
- Tables
- Forms
- Input elements
- Buttons
- Semantic HTML
- Audio/video
- iframe
- Meta tags
- SEO-friendly HTML
- Accessibility basics
- HTML5 APIs introduction

14. CSS3

- CSS syntax

- Selectors
- Colors
- Units
- Typography
- Backgrounds
- Borders
- Box model
- Margin
- Padding
- Display properties
- Positioning
- Overflow
- Z-index
- Pseudo-classes
- Pseudo-elements

Layout

- Flexbox
- CSS Grid

Responsive Design

- Media queries
- Responsive units
- Mobile-first design

Advanced CSS

- Transitions
- Transformations
- Animations
- CSS variables

15. Bootstrap / Tailwind CSS

Students may learn one or both.

Bootstrap

- Grid system
- Containers
- Buttons
- Forms
- Navigation
- Cards
- Modals
- Responsive utilities

Tailwind CSS

- Utility-first CSS
- Responsive classes
- Flex/Grid utilities
- Typography
- Components
- Customization

Part IV — JavaScript Development

16. JavaScript Fundamentals

- Introduction to JavaScript
- Variables
- var, let, const
- Data types
- Operators
- Type conversion
- Conditions
- Loops
- Functions
- Arrays
- Objects
- Strings
- Date/time
- Math object

17. Modern JavaScript — ES6+

- Arrow functions
- Template literals
- Destructuring
- Spread operator
- Rest operator
- Default parameters
- Modules
- Classes
- Inheritance
- Array methods
- map
- filter
- reduce
- find
- some
- every
- Optional chaining
- Nullish coalescing

18. JavaScript DOM & Browser Programming

- DOM
- DOM tree
- Selecting elements
- Modifying elements
- Creating elements
- Event handling
- Forms
- Form validation
- Local storage
- Session storage
- Cookies
- Browser APIs

19. Asynchronous JavaScript

- Synchronous vs asynchronous programming
- Callbacks
- Callback problems
- Promises
- Promise chaining
- `async`
- `await`
- Fetch API
- AJAX
- JSON
- Consuming REST APIs
- Error handling
- Event loop concepts

Part V — React Frontend Development

20. React Fundamentals

- Introduction to React
- SPA architecture
- React project setup
- Vite
- JSX
- Components
- Functional components
- Props
- State
- Events
- Conditional rendering
- Lists
- Keys
- Forms

21. React Hooks

- `useState`
- `useEffect`
- `useRef`
- `useContext`
- `useReducer`
- `useMemo`
- `useCallback`
- Creating custom hooks
- Hook lifecycle concepts

22. React Routing

- React Router
- `BrowserRouter`
- Routes
- `Route`
- `Link`
- `NavLink`

- Nested routes
- Dynamic routing
- Route parameters
- Query parameters
- Protected routes
- 404 pages

23. State Management

- Local state
- Global state
- Context API

Introduction to:

- Redux Toolkit or
- Zustand

Topics:

- Store
- Actions
- Reducers
- Slices
- Selectors
- Async state
- API integration

24. React Forms

- Controlled components
- Uncontrolled components
- Form validation
- React Hook Form
- Validation libraries
- Error messages
- Complex forms
- File upload

25. React API Integration

- Axios
- Fetch
- REST APIs
- GET
- POST
- PUT
- PATCH
- DELETE
- Loading indicators
- Error handling
- Authentication tokens
- API interceptors
- Environment variables
- Reusable API services

Part VI — Python Backend Development

Students should understand both **Django** and modern API development. For an industry-focused course, I would make **Django + Django REST Framework** the main framework and provide **FastAPI** as an additional modern API module.

26. Django Fundamentals

- Introduction to Django
- Django architecture
- MVT pattern
- Installation
- Project creation
- Application creation
- Project structure
- Django settings
- URL routing
- Views
- Templates
- Static files
- Template inheritance
- Template tags
- Template filters

27. Django Models & ORM

- Models
- Fields
- Relationships
- One-to-one
- One-to-many
- Many-to-many
- Migrations
- ORM fundamentals
- CRUD operations
- QuerySets
- Filtering
- Ordering
- Aggregations
- Raw queries
- Model managers
- Database relationships
- Query optimization

28. Django Forms

- Django Forms
- Model Forms
- Form validation
- Custom validators
- CSRF protection
- File uploads
- Image uploads

29. Django Authentication & Authorization

- User model
- Registration
- Login
- Logout
- Password management
- Sessions
- Cookies
- Permissions
- Groups
- Role-based authorization
- Custom user models
- Authentication middleware

30. Django Admin

- Django admin interface
- Registering models
- ModelAdmin
- List views
- Filters
- Search
- Custom admin screens
- User management

31. Django REST Framework

- REST architecture
- API endpoints
- HTTP methods
- Status codes
- Serializers
- ModelSerializer
- APIView
- Generic views
- Mixins
- ViewSets
- Routers
- Pagination
- Filtering
- Searching
- Ordering
- Validation
- Error handling
- API versioning

32. API Authentication & Security

- Basic authentication
- Token authentication
- JWT authentication
- Access tokens
- Refresh tokens
- OAuth concepts
- Role-based access control

- CORS
- CSRF
- Password hashing
- API permissions
- Rate limiting
- Secure API development
- OWASP basics

33. FastAPI

- Introduction to FastAPI
- FastAPI vs Django
- Project setup
- Routing
- Path parameters
- Query parameters
- Request bodies
- Pydantic models
- Validation
- Dependency injection
- Async endpoints
- CRUD APIs
- Authentication
- JWT
- Middleware
- Exception handling
- File uploads
- Background tasks
- Swagger/OpenAPI documentation
- Connecting PostgreSQL
- SQLAlchemy
- Alembic migrations

Part VII — Full-Stack Integration

34. React + Python Backend Integration

Students build a complete SPA architecture:

React → REST API → Django/FastAPI → Database

Topics:

- Frontend/backend communication
- REST architecture
- CORS
- Axios configuration
- Authentication
- JWT storage
- Login/logout
- User registration
- Protected routes
- Authorization
- Error handling
- API response structure

- Form submissions
- File uploads
- Pagination
- Search
- Filtering
- Sorting
- Notifications
- Loading states

35. API Design

- REST principles
- Resource-based URLs
- HTTP methods
- HTTP status codes
- Request/response structures
- DTO/schema concepts
- API validation
- Pagination
- Filtering
- Sorting
- Searching
- API documentation
- Swagger/OpenAPI
- API versioning
- Idempotency
- Rate limiting

Part VIII — Software Engineering

36. Git & GitHub

- Version control
- Git installation
- Repository creation
- `git init`
- `git clone`
- `git add`
- `git commit`
- `git push`
- `git pull`
- Branches
- Merge
- Merge conflicts
- Pull requests
- Code review
- GitHub Issues
- `.gitignore`
- README creation
- GitHub collaboration
- GitHub portfolio development

37. Software Development Practices

- Clean code principles
- Naming conventions
- DRY
- KISS
- YAGNI
- SOLID principles
- Separation of concerns
- Layered architecture
- Repository pattern
- Service layer
- Dependency injection
- Logging
- Configuration management
- Error handling
- Code reviews
- Agile development
- Scrum
- Jira basics

38. Testing

Python Testing

- Unit testing
- unittest
- PyTest
- Fixtures
- Mocking
- API testing

Django Testing

- Model testing
- View testing
- API testing

React Testing

- Component testing concepts
- React Testing Library
- Frontend validation testing

API Tools

- Postman
- Swagger
- API collections
- Automated API testing basics

Part IX — Deployment & DevOps

39. Linux Fundamentals

- Linux architecture
- Basic commands
- Files and directories
- File permissions

- Processes
- Networking basics
- Package management
- Environment variables
- SSH
- Logs

40. Deployment Fundamentals

Understanding:

Development → Testing → Staging → Production

- Deployment architecture
- Application server
- Web server
- Reverse proxy
- Environment variables
- Secrets management
- Static assets
- Database migration
- Logging
- Monitoring

41. Docker

- Containers vs virtual machines
- Docker installation
- Docker images
- Docker containers
- Dockerfile
- Ports
- Volumes
- Networks
- Environment variables
- Docker Compose
- Containerizing React
- Containerizing Django/FastAPI
- Containerizing PostgreSQL
- Multi-container application

42. CI/CD Fundamentals

- Continuous Integration
- Continuous Delivery
- Continuous Deployment
- GitHub Actions
- Build pipelines
- Automated testing
- Deployment pipelines
- Development/staging/production workflows

43. Cloud Deployment

Introduction to:

- AWS / Azure / GCP concepts
- Cloud computing
- Virtual machines
- App hosting
- Object storage
- Managed databases
- Networking
- Domains
- DNS
- SSL/TLS
- HTTPS

Possible deployments:

- AWS
- Azure
- Render
- Railway
- Vercel
- Netlify

Part X — Advanced Full-Stack Development

44. Redis & Caching

- What is caching?
- Redis
- Key-value databases
- Session caching
- Application caching
- Cache invalidation
- Rate limiting
- Performance improvement

45. Background Processing

- Synchronous vs asynchronous work
- Background jobs
- Celery
- Redis/RabbitMQ
- Task queues
- Scheduled jobs
- Email processing
- Report generation

46. WebSockets & Real-Time Applications

- HTTP vs WebSocket
- Real-time communication
- Chat applications
- Notifications
- Django Channels / FastAPI WebSockets
- Frontend WebSocket integration

47. Application Security

- Authentication
- Authorization
- JWT
- OAuth
- CORS
- CSRF
- XSS
- SQL injection
- Password hashing
- Secure cookies
- HTTPS
- Input validation
- Rate limiting
- Secrets management
- OWASP Top 10 awareness

Part XI — AI-Assisted Python Full-Stack Development

This should now be a dedicated module rather than merely teaching students to manually write code.

48. AI-Assisted Development

Tools can include:

- GitHub Copilot
- ChatGPT
- Claude
- Cursor
- AI coding agents

Topics:

- Prompting for code
- Code generation
- Code explanation
- Debugging using AI
- Refactoring
- Unit-test generation
- API creation
- SQL query generation
- React component generation
- Documentation generation
- Code review with AI
- Using AI without blindly accepting generated code
- Security risks of AI-generated code
- Hallucinations in coding assistants
- Context engineering for coding agents

49. Generative AI Integration with Python Applications

An introductory value-added module:

- Generative AI fundamentals
- LLM fundamentals

- Calling an LLM API
- Prompt engineering
- Structured outputs
- Building an AI chatbot
- Integrating AI into Django/FastAPI
- Streaming responses
- Introduction to embeddings
- Introduction to RAG
- AI-powered search
- AI-powered application features

Example:

React → FastAPI → LLM → Database / Knowledge Base

Part XII — Projects

50. Python Mini Projects

Examples:

- Calculator
- Expense tracker
- Contact management system
- Quiz application
- Library management system
- Student management system

51. Frontend Mini Projects

- Portfolio website
- Responsive product page
- Admin dashboard
- Task management UI
- Weather/API application

52. Backend/API Projects

- User authentication API
- Student management REST API
- Product management API
- Blog API
- Employee management system
- E-commerce backend

53. Full-Stack Projects

Students should complete at least **2–3 production-style full-stack projects**.

Project 1 — Task Management System

Features:

- Authentication
- User dashboard
- Tasks
- Categories

- Status
- Search/filtering
- REST APIs

Project 2 — E-Commerce Application

Features:

- Products
- Categories
- Cart
- Authentication
- Orders
- Payments integration concepts
- Admin panel
- Search/filter
- Product images
- REST APIs

Project 3 — Job Portal

Features:

- Candidate registration
- Employer registration
- Jobs
- Applications
- Resume upload
- Search
- Authentication
- Role-based access

54. Capstone Project

A final industry-style project should cover the complete lifecycle:

Requirements → UI Design → Database Design → Frontend → Backend → APIs → Authentication → Testing → Docker → Cloud Deployment

Possible capstone projects:

- Learning Management System
- Hospital Management System
- CRM
- Recruitment Portal
- E-commerce platform
- Food delivery platform
- Project management application
- AI-powered document assistant
- AI-enabled customer support system

Part XIII — Career & Placement Preparation

55. GitHub Profile Building

- Professional GitHub profile
- GitHub profile README
- Repository organization

- Professional README files
- Project screenshots
- API documentation
- Commit practices
- Portfolio projects
- Open-source basics

56. Resume & LinkedIn Preparation

- Python full-stack developer resume
- ATS-friendly resume
- Project descriptions
- Technical skills presentation
- LinkedIn optimization
- GitHub portfolio integration

57. Interview Preparation

Python

- Core Python interview questions
- OOP
- Collections
- Functions
- Decorators
- Generators
- Exception handling

SQL

- Queries
- Joins
- Subqueries
- Indexes
- Normalization

Django/FastAPI

- Framework architecture
- ORM
- REST APIs
- Authentication
- Middleware
- Security

React

- Components
- State
- Props
- Hooks
- Routing
- State management

Web

- HTTP
- REST
- Cookies

- Sessions
- JWT
- CORS

DSA

- Coding problems
- Arrays
- Strings
- Hashing
- Searching
- Sorting
- Basic trees/graphs

System Design Basics

- Three-tier architecture
- API design
- Database selection
- Caching
- Load balancing
- Scalability fundamentals

58. Mock Interviews & Coding Practice

- Technical mock interviews
- HR interviews
- Coding assessments
- Python coding challenges
- SQL challenges
- API design exercises
- Debugging exercises
- Project explanation practice
- System design discussions

Recommended Technology Stack

- Layer — Technologies
- Programming — Python
- Frontend Fundamentals — HTML5, CSS3, JavaScript
- CSS Framework — Bootstrap / Tailwind
- Frontend Framework — React
- State Management — Context API, Redux Toolkit / Zustand
- Backend — Django, Django REST Framework
- Modern API Framework — FastAPI
- Relational Database — PostgreSQL / MySQL
- NoSQL — MongoDB
- ORM — Django ORM, SQLAlchemy
- API — REST, JSON, Swagger/OpenAPI
- Authentication — Sessions, JWT, OAuth basics
- Version Control — Git, GitHub
- API Testing — Postman
- Testing — PyTest, React Testing Library
- Cache — Redis
- Background Jobs — Celery

- Containers — Docker, Docker Compose
- CI/CD — GitHub Actions
- Cloud — AWS / Azure fundamentals
- AI-Assisted Development — GitHub Copilot, ChatGPT, Claude/Cursor
- GenAI Integration — LLM APIs, embeddings, introductory RAG

Recommended Learning Sequence

Python → OOP → DSA → SQL → HTML → CSS → JavaScript → React → Django → Django REST Framework → FastAPI → Full-Stack Integration → Git/GitHub → Testing → Docker → CI/CD → Cloud → AI-Assisted Development → Capstone Project → Interview Preparation

For an institute-level course, I would position it as an **“AI-Assisted Python Full-Stack Development Course”** rather than just “Python Full-Stack Development.” It differentiates the curriculum by teaching students not only Python, Django, FastAPI and React, but also how modern developers work with coding assistants and integrate generative AI into applications.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.